Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Модели данных и системы управления базами данных»

ОТЧЕТ
к лабораторной работе №5
на тему:
«REDIS, ОЧЕРЕДИ»
БГУИР 6-05-0612-02 67

Выполнил студент группы 353503
КОХАН Артём Игоревич

_____
(дата, подпись студента)

Проверил ассистент каф. информатики
ИОВИЧ Евгения Александровна

_____
(дата, подпись преподавателя)

Минск 2026

1 ИНДИВИДУАЛЬНОЕ ЗАДАНИЕ

Лабораторная работа предназначена для определения возможных ролей Redis в архитектуре: таких как очереди, rate limiting, real-time, persistance, масштабирование (на базовом уровне).

1. Необходимо реализовать очередь задач с использованием Lists
   - 1.1 Спроектировать очередь фоновых задач:
     - ключ очереди, например queue:tasks;
     - формат задачи (JSON-строка как значение, ID как часть payload).
   - 1.2 Реализовать:
     - producer: добавляет задания в очередь с помощью LPUSH/RPUSH;
     - consumer (воркер): читает задания BRPOP/BLPOP, обрабатывает, логирует результат.
   - 1.3 Обработать как минимум два типа задач (например, отправка email / обработка заказа / запись лога).

2 реализовать **ограничение количества запросов к API** для одного пользователя/токена, например: не более 100 запросов в 10 минут.

- 2.1 **Простой счетчик с TTL**:
  - ключ rate:{user_id}:{window_start_timestamp};
  - при каждом запросе:
    - INCR key;
    - если INCR вернул 1 — поставить EXPIRE key window_size;
    - если значение > limit → запрос отклонить.
- **2.2 Скользящее окно**
  - хранить timestamps в List или Sorted Set и периодически очищать старые;
  - реализовать логику через Lua-скрипт для атомарности.

2 КРАТКИЕ ТЕОРИТИЧЕСКИЕ СВЕДЕНИЯ

В контексте данной работы рассматриваются следующие основные роли Redis в архитектуре приложений:

1 Очередь задач: Используя структуру данных List и команды LPUSH/RPUSH (для добавления) и BRPOP/BLPOP (для блокирующего чтения), Redis может эффективно выступать в роли простого брокера сообщений или очереди фоновых задач. Это позволяет организовать асинхронную обработку, повышая отзывчивость основного приложения и обеспечивая надежную доставку заданий между производителями (producers) и потребителями (workers).

2 Ограничение скорости запросов (Rate Limiting): Redis идеально подходит для реализации механизмов контроля частоты запросов (rate limiting) благодаря высокой скорости работы и поддержке атомарных операций. Простой счетчик с TTL: На каждый интервал времени (окно) создается отдельный ключ-счетчик. Операции INCR и EXPIRE позволяют отслеживать количество запросов и автоматически сбрасывать счетчик по истечении окна. Скользящее окно (Sliding Window): Более точный метод, использующий Sorted Set или List для хранения временных меток запросов. Он позволяет ограничивать частоту запросов в любом скользящем интервале, а не в фиксированных временных блоках. Атомарность достигается с помощью Lua-скриптов.

3 Атомарность и консистентность: Redis обеспечивает атомарное выполнение отдельных команд и транзакций (pipeline, MULTI/EXEC), а также выполнение Lua-скриптов. Это важно для реализации корректных механизмов очередей и rate limiting в многопоточной среде.

3 ВЫПОЛНЕНИЕ РАБОТЫ

Для реализации очереди задача с использованием List в Redis определен единый ключ очереди queue:tasks. Формат задачи – JSON-строка, содержащая идентификатор (id), тип задачи (type: send_email, process_order, write_log), полезную нагрузку (payload) и метку времени создания (created_at). Для работы с задачами создан класс Task с методами сериализации (to_json) и десериализации (from_json).

Создан класс TaskProducer. Он подключается к Redis и с помощью метода lpush добавляет задачи в начало списка queue:tasks. Реализованы методы-генераторы для создания трех типов задач: отправки email, обработки заказа и записи лога. Каждая задача имеет уникальный ID.

Создан класс TaskConsumer. Он использует команду brpop для блокирующего чтения задачи из конца очереди. При получении задачи JSON десериализуется в объект Task, и в зависимости от ее типа вызывается соответствующий метод обработки (_process_email_task, _process_order_task, _process_log_task). Имитируется задержка обработки. Воркер работает в бесконечном цикле.

Запущен отдельно производитель, который создает и помещает в очередь три примера задач, как на рисунке 3.1.

```
/home/koxan/SQL/LAB_5_2s/execute/.venv/bin/python /home/koxan/SQL/LAB_5_2s/execute/producer.py
Task ecc7e68a-195d-4250-ab9c-3da9cd88e47a added to queue. Queue length: 1
Task f0c91434-2d4f-4111-ad16-380e0055fd32 added to queue. Queue length: 2
Task be3ef8fc-0d49-464b-abfb-60aba847ae1b added to queue. Queue length: 3

Process finished with exit code 0
```

Рисунок 3.1 – Добавление задач в очередь

Убедиться в том, что задачи были действительно добавлены в очередь и находятся в базе данных (рисунок 3.2), важно для проверки корректности TaskProducer.

Рисунок 3.2 – Список задач в базе данных

Помимо производителя, запущен также потребитель, который последовательно их извлекает и обрабатывает, выводя информацию в консоль (рисунок 3.3).

```
/home/koxan/SQL/LAB_5_2s/execute/.venv/bin/python /home/koxan/SQL/LAB_5_2s/execute/consumer.py
Consumer started. Listening on queue: queue:tasks

Processing task ecc7e68a-195d-4250-ab9c-3da9cd88e47a of type send_email
Sending email to user@example.com
Subject: Welcome!
Body: Thank you for registering!...
Email sent successfully to user@example.com

Processing task f0c91434-2d4f-4111-ad16-380e0055fd32 of type process_order
Processing order 1001
Customer: 12345
Amount: $99.99
Order 1001 processed successfully

Processing task be3ef8fc-0d49-464b-abfb-60aba847ae1b of type write_log
Writing log: [INFO] auth-service - User login successful
Log written successfully
```

Рисунок 3.3 – Извлечение и обработка задач потребителем

Для реализации ограничения количества запросов реализован простой счётчик в методе check_simple. Ключ формируется по шаблону rate:{user_id}:{window_start}, где window_start – целая часть от деления текущего времени на длину окна. При каждом запросе выполняется атомарная операция: INCR для увеличения счетчика и EXPIRE для установки TTL, равного длине окна (если счетчик был равен 1). Если значение счетчика превышает лимит, запрос отклоняется, собственно, всё как на рисунке 3.4.

```
Запрос 1: ✓ Разрешено
Запрос 2: ✓ Разрешено
Запрос 3: ✓ Разрешено
Запрос 4: ✓ Разрешено
Запрос 5: ✓ Разрешено
Запрос 6: ✓ Разрешено
Запрос 7: ✓ Разрешено
Запрос 8: ✓ Разрешено
Запрос 9: ✓ Разрешено
Запрос 10: ✓ Разрешено
Запрос 11: ✗ Отклонено
Запрос 12: ✗ Отклонено
Запрос 13: ✗ Отклонено
Запрос 14: ✗ Отклонено
Запрос 15: ✗ Отклонено
```

Рисунок 3.4 – Ограничение количества запросов счётчиком

Прошлый рисунок не придаёт полной ясности, поэтому следует сослаться ещё на рисунок 3.5, где можно увидеть, что попадает в базу данных.

Рисунок 3.5 – Значение счётчика в базе данных

Скользящее окно на Sorted Set реализовано в методе check_sliding. Используется ключ rate_slide:{user_id} и структура Sorted Set, где в качестве score хранится временная метка запроса (timestamp). Логика инкапсулирована в Lua-скрипт для гарантии атомарности. В результате выполнения функции демонстрации получим Sorted Set, представленный на рисунке 3.6

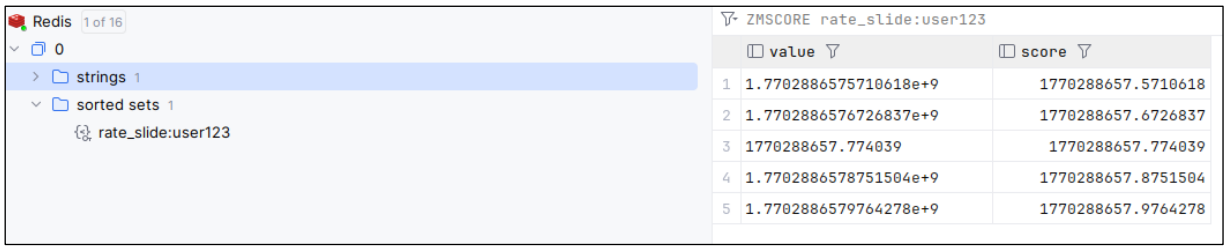

Рисунок 3.6 – Скользящее окно на Sorted Set

ВЫВОД

В ходе лабораторной работы продемонстрировано использование структуры List в качестве простой, но эффективной очереди сообщений. Реализация показала возможность организации асинхронной обработки фоновых задач (отправка email, обработка заказов, логирование).

Реализованы и сравнены два классических алгоритма ограничения запросов. Простой счетчик с TTL эффективен и прост в реализации и алгоритм скользящего окна, реализованный на Sorted Set с атомарным Lua-скриптом.

Redis, работающий с данными в памяти, показал себя как идеальный инструмент для таких сценариев благодаря своей исключительной скорости. Redis предоставляет минималистичные, но мощные примитивы (List, Sorted Set, ключи с TTL), которые в сочетании с различными клиентскими библиотеками позволяют быстро реализовывать сложные паттерны распределенных систем без привлечения тяжеловесных специализированных брокеров или систем координации. Таким образом, Redis является универсальным инструментом, который может выполнять не только функцию кэша, но и выступать в качестве полноценного компонента для организации асинхронной коммуникации, управления нагрузкой и обеспечения контроля доступа в современных распределенных приложениях.